# JDBC

Java Database Connectivity

# JDBC

Es un conjunto de clases e interfaces Java que permiten la manipulación de sentencias SQL de una fuente de datos ( base de datos ).

La API de JDBC define un conjunto de clases e interfaces que proporcionan toda la funcionalidad que el acceso a base de datos requiere, tal como la ejecución de consultas SQL o el tratamiento de resultados

Todo el conjunto de clases e interfaces de JDBC se encuentra dentro del paquete: *java.sql* y *javax.sql*

# Controlador

También llamado drivers JDBC y son adaptadores de Cliente, es decir, se instala en la máquina y no el servidor.

Es una implementación de varias interfaces especificadas en los paquetes

Se usa para conectar una aplicacion en java con una Base de Datos.

# Clases e Interfaces

## Conexión:

- **DriveManager**: Se usa para cargar un driver
- **Connection** : Se usa para establecer conexiones con las bases de datos
- **DatabaseMetaData:** Información sobre la base de datos en general

# Clases e Interfaces

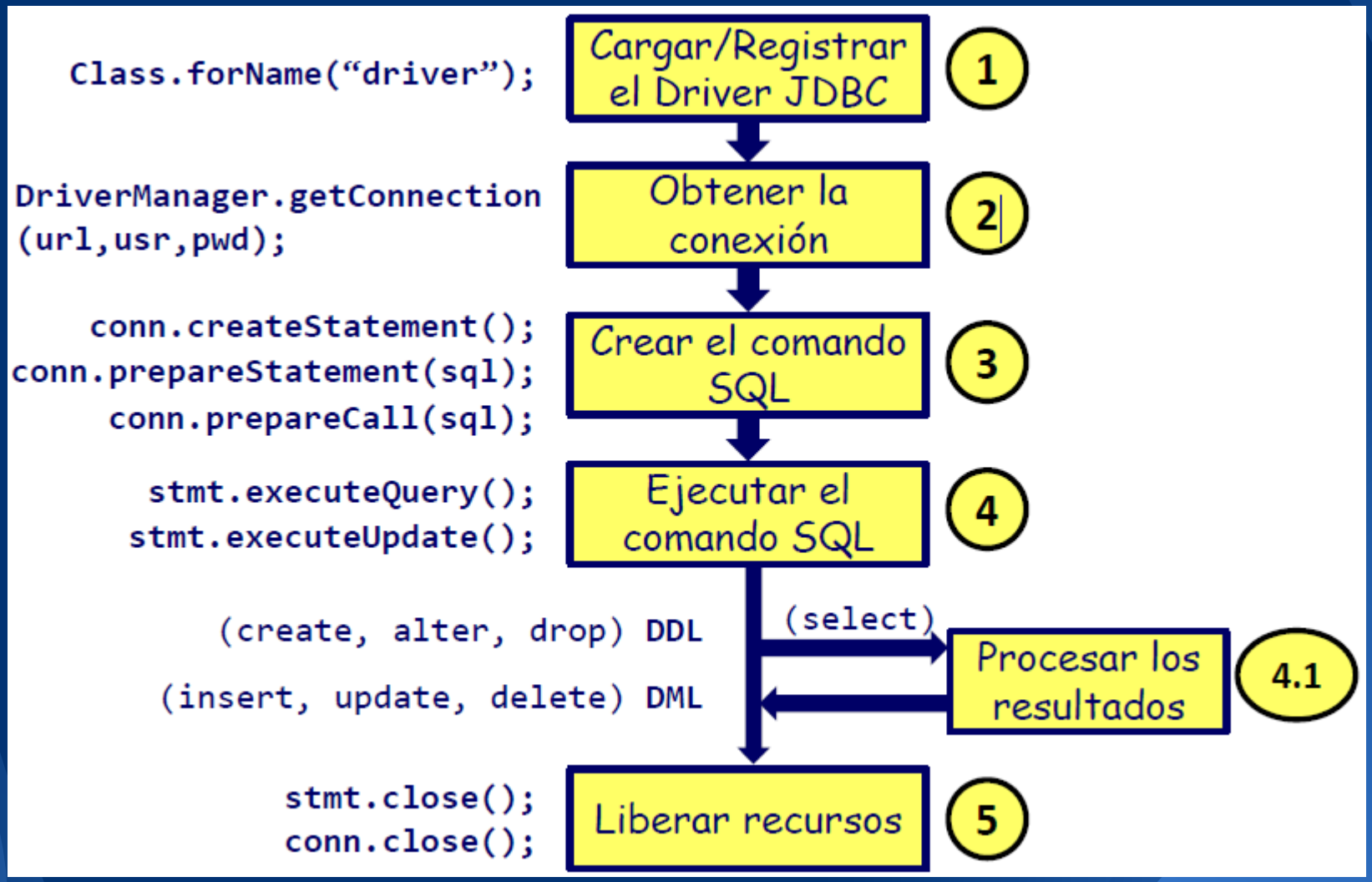
## Comandos:

- **Statement**: Se usa para crear sentencias SQL y enviarlas a las BD
- **PreparedStatment** : Similar a Statement con la diferencia que se protege de inyecciones

## Resultados:

- **ResulSet**: Recibe el resultado de las consultas
- **ResultSetMetaData:** Información propiedades de un ResultSet

# Pasos para utilizar JDBC en Java



# Conexión

```
public void conectar(String servidor, String bd, String usuario, String password) throws ClassNotFoundException, SQLException{

    Class.forName("com.mysql.jdbc.Driver");

    Connection conn = DriverManager.getConnection("jdbc:mysql://"+servidor+"/"+bd,usuario,password);
}
```

**servidor:** *localhost* ó *127.0.0.1*

(ubicación donde se encuentra el servidor de la base de datos, puede ser cambiado por la ip si el servidor se encuentra en otro equipo en red)

**bd:** *test* (nombre de la base de datos)

**usuario:** *nelsonaph*

**password:** *********

# CRUD

**C**reate
**R**ead
**U**pdate
**D**elete

# CRUD

Para efectuar las sentencias SQL es necesario utilizar la interface Statement que provee métodos para realizar esas tareas

- **executeUpdate**
  Retorna un número entero indicando la cantidad de registros afectados UPDATE-INSERT-DELETE

- **executeQuery**
  Regresa un conjunto de resultados que se almacenan en un objeto *ResultSet*

# Create

```
public void insertar(String user, String pass){
    try {
        //Se inicia la declaracion que se va a realizar
        consultados = conn.createStatement();
        //Se ejecutan los comandos para actualizar la tabla Usuarios (en este caso crear)
        consultados.executeUpdate("insert into Usuarios values("+pass+" , '"+user+"')");
    } catch (SQLException ex) {
        Logger.getLogger(Servicio.class.getName()).log(Level.SEVERE, null, ex);
    }

}
```

Instanciamos a ***consultados*** que es un objeto de la clase *Statement*

Realizamos la consulta
`consultados.executeUpdate(" CONSULTA SQL ")`

# Statement y PreparedStatement

El problema con Statment sucede cuando la consulta se realiza dentro de un ciclo y varia solo en unos valores:
*stmt.executeQuery("select * from Usuarios where id=" + i)*

En cambio, con el PreparedStatement se declara la sentencia en general y si se ejecuta varias veces solo se cambian los parámetros de entrada sin tener que volver a declarar toda la sentencia

```
Statement ipStmt = conn.prepareStatement(
"INSERT INTO contactos(nombre,telefono,email)
VALUES(?,?,?)");
ipStmt.setString(1,"Juan");
ipStmt.setString(2,"01667-7505816");
ipStmt.setString(3,"juan@hotmail.com");
int i = ipStmt.executeUpdate();
```

Parámetros IN

# Read

```
//Se prepara la declaraci0n que se va a hacer para consultar en la base de datos
consulta = conn.prepareStatement("SELECT id, usuario FROM Usuarios where usuario='"
        +user+"' and id='"+pass+"'");
//Se ejecuta la declaraci0n, se consultan los datos que quedan
//almacenados en el atributo datos
datos=consulta.executeQuery();
```

***consulta*** es un objeto de ***PreparedStatement*** la cual es similar a Statement con la diferencia que es más seguro. porque precompila el query.

`datos=consulta.executeQuery();`

La consulta nos genera un resultado tipo *Resulset* y lo podemos guardar en "*datos*".

Imprimir la consulta:

```
System.out.print(datos.getString("usuario"));
```

Cuando se lanza un método getXXX sobre un objeto ResultSet, el driver JDBC convierte el dato que se quiere recuperar a el tipo Java especificado y entonces devuelve un valor Java adecuado.

## Tipos de datos y conversiones.

- El mapeo o conversión es la siguiente:

| Tipos JDBC | Tipos Java | Tipos JDBC | Tipos Java |
|---|---|---|---|
| BIGINT | getLong() | LONGVARCHAR | getString() |
| BINARY | getBytes() | NUMERIC | getBigDecimal() |
| BIT | getBoolean() | OTHER | getObject() |
| CHAR | getString() | REAL | getFloat() |
| DATE | getDate() | SMALLINT | getShort() |
| DECIMAL | getBigDecimal() | TIME | getTime() |
| DOUBLE | getDouble() | DATESTAMP | getTimeStamp() |
| FLOAT | getDouble() | TINYINT | getByte() |
| INTEGER | getInt() | VARBINARY | getBytes() |
| LONGVARBINARY | getBytes() | VARCHAR | getString() |

# Update

```
public void actualizar(String user, String pass, String newpass){
    try {
        //Se inicia la declaracion que se va a realizar
        consultados = conn.createStatement();
        //Se ejecutan el comando para actualizar las tablas Estudiante
        consultados.executeUpdate("update Usuarios set id="+newpass+","
                +" usuario='"+user+"' where id='"+pass+"'");

        //Se manejan excepciones
    } catch (SQLException ex) {
        Logger.getLogger(Servicio.class.getName()).log(Level.SEVERE, null, ex);
    }

}
```

# Delete

```
public void eliminar(String user, String pass){
    try {
        //Se inicia la declaracion que se va a realizar
        consultados = conn.createStatement();
        //Se ejecutan los comandos para actualizar la tabla (en este caso eliminar)
        consultados.executeUpdate("delete from Usuarios where usuario='"+user+"' and id='"+pass+"'");
        //Se manejan las excepciones
    } catch (SQLException ex) {
        Logger.getLogger(Servicio.class.getName()).log(Level.SEVERE, null, ex);
    }

}
```

# Manejo de Excepciones

La mayor parte de las operaciones que nos proporciona el API JDBC lanzarán la excepción java.sql.SQLException en caso de que se produzca algún error a la base de datos, pueden ser errores de conexión, sentencias sql incorrectas, falta de privilegios, etc.

# Manejo de Excepciones

Es necesario dar un tratamiento adecuado a estas excepciones y encerrar todo el código JDBC entre bloques try/catch.

JDBC da acceso a la información de un fallo proporcionando dos niveles de condiciones del error: **SQLException** y **SQLWarning**

# Manejo de Excepciones

SQLExceptions: Son las excepciones de Java que, si no manejó, terminarán la aplicación.

```
try {
   con.setAutoCommit(false) ;
   stmt.executeUpdate("CREATE TABLE Venta (nombreBar VARCHAR2(40)," +
           " marcaCerveza VARHAR2(40), precio REAL)") ;
   stmt.executeUpdate("INSERT INTO Venta VALUES " +
                      "('Bar Ato', 'Mellir', 1200)") ;
   con.commit() ;
   con.setAutoCommit(true) ;

}catch(SQLException ex) {
   System.err.println("SQLException: " + ex.getMessage()) ;
   con.rollback() ;
   con.setAutoCommit(true) ;
}
```

# Manejo de Excepciones

SQLWarnings: Representan errores no fatales o condiciones inesperadas, pueden ignorarse.

```
ResultSet rs = stmt.executeQuery("SELECT nombreBar FROM Venta") ;
SQLWarning warn = stmt.getWarnings() ;
if (warn != null)
   System.out.println("Message: " + warn.getMessage()) ;
SQLWarning warning = rs.getWarnings() ;
if (warning != null)
   warning = warning.getNextWarning() ;
if (warning != null)
   System.out.println("Message: " + warn.getMessage()) ;
```

# Referencias


http://eisc.univalle.edu.co/materias/BD/IntroduccionAlJDBC.pdf

http://www.codigofantasma.com/blog/conectar-java-con-mysql-usando-jdbc/

http://docs.oracle.com/javase/tutorial/jdbc/basics/gettingstarted.html

http://www.slideshare.net/roramas/curso-bsico-de-jdbc

# GRACIAS

Nelson Portilla
Juan Diego Prado
Fernando Sanchez

Universidad del Valle - 2013